

Tema 6: Shell Scripting Basico

Vamos a aprender a escribir scripts de shell. En esta asignatura usamos sh, que es el shell POSIX estandar, no bash. La primera linea de todo script debe ser almohadilla exclamacion barra bin barra sh. Esto es lo que se llama shebang, y le dice al sistema que interprete para usar.

Parametros posicionales

Los scripts reciben argumentos desde la linea de comandos. Dolar cero es el nombre del script. Dolar uno, dolar dos, etcetera, son los argumentos individuales. Dolar almohadilla es el numero de argumentos, sin contar el nombre del script. Dolar arroba es todos los argumentos como palabras separadas, y dolar asterisco es todos los argumentos como una sola cadena.

La diferencia entre dolar arroba y dolar asterisco es importantisima: cuando los pones entre comillas, dolar arroba mantiene cada argumento como una palabra separada, mientras que dolar asterisco los junta todos en una sola cadena. Esto importa cuando los argumentos contienen espacios.

El comando shift desplaza los argumentos hacia la izquierda. Despues de shift, dolar uno contiene lo que antes era dolar dos, dolar dos lo que era dolar tres, y asi sucesivamente. Dolar almohadilla se decrementa. Es util para procesar argumentos en un bucle.

Ejemplo de examen: un script con un for que itera sobre dolar asterisco, dentro del for hace echo de dolar uno y shift. Si lo ejecutas con tres argumentos, imprime cada argumento en una linea. Esto funciona porque en cada iteracion, shift mueve los argumentos y dolar uno cambia.

Ejecucion de scripts

Hay dos formas de ejecutar un script: como un subshell, o en el mismo shell. Si ejecutas con barra punto script, o con sh script, se crea un nuevo proceso hijo que ejecuta el script. Las variables definidas en el script no afectan al shell padre. Si usas punto espacio script, o source script, el script se ejecuta en el mismo shell, y las variables si afectan al shell actual.

Esta diferencia es muy importante: las variables del shell no se heredan a los hijos a menos que uses export. Si defines una variable en tu shell actual y luego ejecutas otro script como subshell, esa variable no existe en el script hijo. Esta es una pregunta clasica de examen.

Sustitucion de comandos

Para capturar la salida de un comando en una variable, usas la sustitucion de comandos. La sintaxis moderna es dolar parentesis abierto comando parentesis cerrado. La sintaxis antigua, que tambien funciona pero es peor, usa acentos graves. El resultado es que la salida del comando se sustituye en el lugar donde escribiste la sustitucion.

Por ejemplo, variable igual a dolar parentesis date parentesis cerrado almacena la salida del comando date en la variable. Luego puedes usar la variable normalmente.

Condicionales con if y test

La sentencia if evalua un comando y, si su codigo de salida es cero, que significa exito, ejecuta el bloque then. Si no, ejecuta el bloque else si lo hay. El bloque termina con fi.

Para hacer comparaciones usamos el comando test, que tambien se puede escribir como corchete abierto condicion corchete cerrado. Hay tres tipos de comparaciones.

Para ficheros: test menos f comprueba si es un fichero regular, test menos d comprueba si es un directorio, test menos e comprueba si existe, test menos r comprueba si tiene permiso de lectura, test menos x comprueba si es ejecutable. El signo de exclamacion niega la condicion: test exclamacion menos f significa que no es un fichero regular.

Para cadenas: test cadena uno igual cadena dos comprueba igualdad, test cadena uno exclamacion igual cadena dos comprueba desigualdad, test menos z cadena comprueba si esta vacia, test menos n cadena comprueba si no esta vacia.

Para numeros enteros: menos eq es igual, menos ne es distinto, menos lt es menor que, menos le es menor o igual, menos gt es mayor que, menos ge es mayor o igual. Atencion: para numeros se usa menos eq, no igual. El igual es para cadenas.

La sentencia case

Case compara una variable contra varios patrones. La sintaxis es case dolar variable in, luego cada patron seguido de parentesis cerrado, las acciones, y doble punto y coma. Se cierra con esac.

Los patrones pueden incluir caracteres comodin como asterisco. Es muy util para procesar opciones de linea de comandos o para comprobar extensiones de ficheros.

Bucles while y for

El bucle while ejecuta un bloque mientras un comando devuelva exito. La sintaxis es while comando, do, cuerpo, done. Un uso tipico es while read linea, que lee un fichero linea por linea hasta el final.

El bucle for itera sobre una lista de valores. La sintaxis es for variable in lista, do, cuerpo, done. Ejemplo: for f in dolar arroba, do, echo dolar f, done. Esto imprime cada argumento del script.

Combinacion tipica: listar ficheros txt de un directorio y procesarlos: for f in dolar parentesis ls asterisco punto txt parentesis cerrado, do, echo procesando dolar f, done.

El comando read

Read lee una linea de la entrada estandar y la almacena en una o mas variables. Si pones read a b, y la entrada es hola mundo cruel, a toma hola y b toma mundo cruel, es decir, la ultima variable recibe todo lo que queda.

El separador de campos por defecto es el espacio y el tabulador. Puedes cambiarlo modificando la variable IFS. Por ejemplo, IFS igual a dos puntos permite leer campos separados por dos puntos, que es útil para parsear ficheros como passwd.

Para leer un fichero línea por línea, el patrón es: `while IFS igual a cadena vacia read menos r linea, do, echo dolar linea, done` menor que fichero. La opción `menos r` evita que `read` interprete las barras invertidas.

Funciones en shell

Las funciones se definen con nombre parentesis llaves, cuerpo, cierre de llave. Los argumentos se acceden igual que en el script principal: `dolar uno, dolar dos, etcetera`, pero son locales a la función.

La función devuelve un código de salida con `return`, no el resultado de un cálculo. El código de salida cero significa éxito, y cualquier otro valor significa error. Si quieres que la función devuelva un valor de texto, la función lo imprime con `echo` y el llamador captura la salida con sustitución de comandos.

Aritmetica

Para operaciones aritmeticas en shell, usamos `dolar parentesis doble expresion parentesis doble`. Por ejemplo, `resultado igual a dolar parentesis doble tres mas cuatro parentesis doble` asigna siete a `resultado`. Dentro de la doble parentesis puedes usar suma, resta, multiplicación, división entera y módulo.

Errores comunes en shell

Primer error: olvidar las comillas alrededor de las variables. Si una variable contiene espacios y no la entrecomillas, se divide en múltiples palabras. Siempre pon comillas: `echo comilla dolar variable comilla`.

Segundo error: confundir las comparaciones numéricas y de cadena. Para comparar números usa `menos eq`, `menos lt`, etcetera. Para comparar cadenas usa `igual` y exclamación igual.

Tercer error: olvidar cerrar el `if` con `fi`, el `case` con `esac`, o el `do` con `done`.

Cuarto error: asumir que las variables del padre se heredan al hijo. Solo se heredan si usas `export`.